Lab Report: Constructing a Rule-Based Agent

1. Title

Constructing a Rule-Based Agent for a Small Problem Domain

---

2. Objective

- To understand the concept of rule-based agents in artificial intelligence.
- To design and implement a simple rule-based agent that can make decisions based on a set of predefined rules.
- To analyze the performance and behavior of the agent in a controlled environment.

---

3. Theory

A **rule-based agent** is an artificial intelligence system that **follows a set of rules** to make decisions. These agents operate in a specific environment and act according to "if-then" rules:

Basic structure of a rule-based agent:

1. **Perception:** The agent perceives its environment.
2. **Rule Matching:** It matches its perception against a set of rules.
3. **Action Selection:** Executes the action corresponding to the matched rule.

Example:

- If the room is dark → Turn on the light
- If the door is closed → Open the door

Rule-based systems are useful for **simple decision-making problems** and can be implemented with simple programming logic.

---

4. Materials Required

- Computer with Python IDE (like PyCharm, VS Code, or Jupyter Notebook)

- Python programming language
- Basic knowledge of “if-else” statements and functions
- Optional: Flowchart tool for visualizing rules

---

5. Problem Domain

For this experiment, we will use a **simple home automation system** as the problem domain:

- **Environment:** A room with sensors for light, temperature, and door status
- **Goal:** The agent will take actions such as turning on/off lights, opening/closing doors, and adjusting temperature.

---

6. Procedure

1. Define the environment and the state variables.
    - `light`: on/off
    - `temperature`: hot/cold
    - `door`: open/closed
2. Define rules for the agent:
    - If `light = off` → turn on light
    - If `temperature = hot` → turn on fan
    - If `door = closed` → open door
3. Implement the rules in Python using if-else statements.
4. Test the agent with different input scenarios to see if it acts correctly.

---

7. Implementation (Python Code Example)

```
# Rule-Based Agent for Home Automation

def rule_based_agent(light, temperature, door):
    actions = []

    # Rule 1: Light control
    if light == "off":
        actions.append("Turn on the light")
    else:
        actions.append("Light is already on")

    # Rule 2: Temperature control
    if temperature == "hot":
        actions.append("Turn on the fan")
```

```
    elif temperature == "cold":
        actions.append("Turn on the heater")
    else:
        actions.append("Temperature is comfortable")

    # Rule 3: Door control
    if door == "closed":
        actions.append("Open the door")
    else:
        actions.append("Door is already open")

    return actions

# Test scenarios
scenario1 = rule_based_agent(light="off", temperature="hot", door="closed")
scenario2 = rule_based_agent(light="on", temperature="cold", door="open")

print("Scenario 1 Actions:", scenario1)
print("Scenario 2 Actions:", scenario2)
```

---

8. Results

Scenario	Light	Temperature	Door	Actions Taken
1	off	hot	closed	Turn on light, Turn on fan, Open door
2	on	cold	open	Light is already on, Turn on heater, Door is already open

The agent correctly performed actions according to the rules defined.

---

9. Discussion

- The agent successfully made decisions using a predefined set of rules.
- Rule-based agents are simple to design and understand but can become **unmanageable** as the number of rules increases.
- The system works well for small, well-defined domains but may struggle in dynamic or complex environments where rules conflict.

---

10. Conclusion

- A rule-based agent was successfully implemented for a small problem domain.
- The agent's behavior accurately reflected the rules set for the environment.
- Rule-based agents are a good starting point for understanding AI decision-making in controlled scenarios.

11. Precautions

- Ensure all possible states are covered in rules to avoid undefined behavior.
- Test the agent with multiple scenarios to validate correctness.
- Keep rules simple to avoid conflicts.